

Introduction to Databases

- The database serves as the backbone of the Smart Healthcare Management Platform. It ensures the preservation, consistency, and integrity of all medical, financial, and administrative data, while providing a real-time, secure connection between various hospital departments (doctors, patients, laboratories, radiology, pharmacy, and the reception desk). The data architecture is designed to achieve maximum query efficiency and to guarantee system stability under the heavy data loads generated by daily transactions within the medical facility.

2. DBMS Selection & Development Environment

The reliable MySQL Relational Database Management System (RDBMS) was selected to build the platform's database. This engineering choice is backed by several technical reasons:

- **Support for Complex Transactions (ACID Properties):** MySQL ensures data integrity and consistency during concurrent modification processes, such as managing simultaneous appointment bookings to prevent double-booking or recording precise financial payments.
- **High Compatibility:** MySQL integrates seamlessly with backend development frameworks and programming languages (whether utilizing PHP, Python, or Node.js).
- **Speed and Performance:** It offers high capability in processing complex queries and efficiently executing JOIN operations across large-scale tables.

Part 1: Structure of the Appointments Table (appointments)

The table is designed to cover all booking details within the hospital system and contains the following fields:

- **Primary and Foreign Keys:** `id` (Appointment ID), `patient_id` (Patient ID), `doctor_id` (Doctor ID), and `department_id` (Department ID).
- **Timing:** `appointment_date` (Date of the appointment), `start_time` (Start time), and `end_time` (End time).
- **Appointment Status and Queue:** `status` (The booking status, which defaults to 'pending' or updates to 'confirmed' or 'cancelled'), and `queue_number` (The patient's turn or queue number).
- **Medical and Financial Data:** `treatment` (Type of medical procedure, e.g., consultation, check-up, follow-up), `notes` (Doctor's notes), and `fee` (Consultation/visiting fee).
- **Communication and Documentation:** `phone_number` (Contact number), as well as timestamps for automated confirmation and email tracking (`auto_confirmed_at`, `confirmation_email_sent_at`, `created_at`, `updated_at`).

Part 2: Analysis of the Inserted Data (Data Dumping)

Based on the provided data sample, we can extract the following key indicators to support the database and backend presentation of your project:

1. Appointment Status (`status`):

- a. There are **confirmed** appointments, such as appointment ID (2) for patient ID (2) with doctor ID (2) on 2026-04-22.
- b. There are **cancelled** appointments, such as appointment IDs (51) and (52).

2. Types of Medical Procedures (`treatment`): The medical services vary across the database to include:

- a. **Cardiology Consultations & Diagnostics:** Cardiac Consultation, Cardiac Check-Up, and Heart Disease Diagnostics (heavily concentrated in department ID 2, with doctor IDs 2, 7, and 24).
- b. **General Consultations & Follow-ups:** General Consultation and Follow-Up Visit.

c. **Chronic Condition Management:** Hypertension Management.

3. Pricing and Fees (fee):

- a. Financial values vary significantly; for instance, cardiology consultations (Department 2) generally range between **300 to 338 currency units/EGP**.
- b. General consultations and follow-ups show a wide range of pricing (some are very low or zero, such as 5, 8, or 0.00 in appointment IDs 56, 92, and 164, which might indicate health insurance coverages or free cases, while others reach 290 and 340).

4. Doctor Notes (notes):

- a. Contains standard English seed data, such as "Seeded sample appointment".
- b. Contains user-entered Arabic characters (e.g., in appointment IDs 46 and 47), which proves that the database correctly supports Arabic character encoding (utf8mb4_unicode_ci) to handle regional user inputs perfectly.

in general:

3. Entity-Relationship Diagram (ERD)

The structural design of the database converts the system's functional requirements into concrete entities and logical relationships. The architecture is categorized into the following core tables:

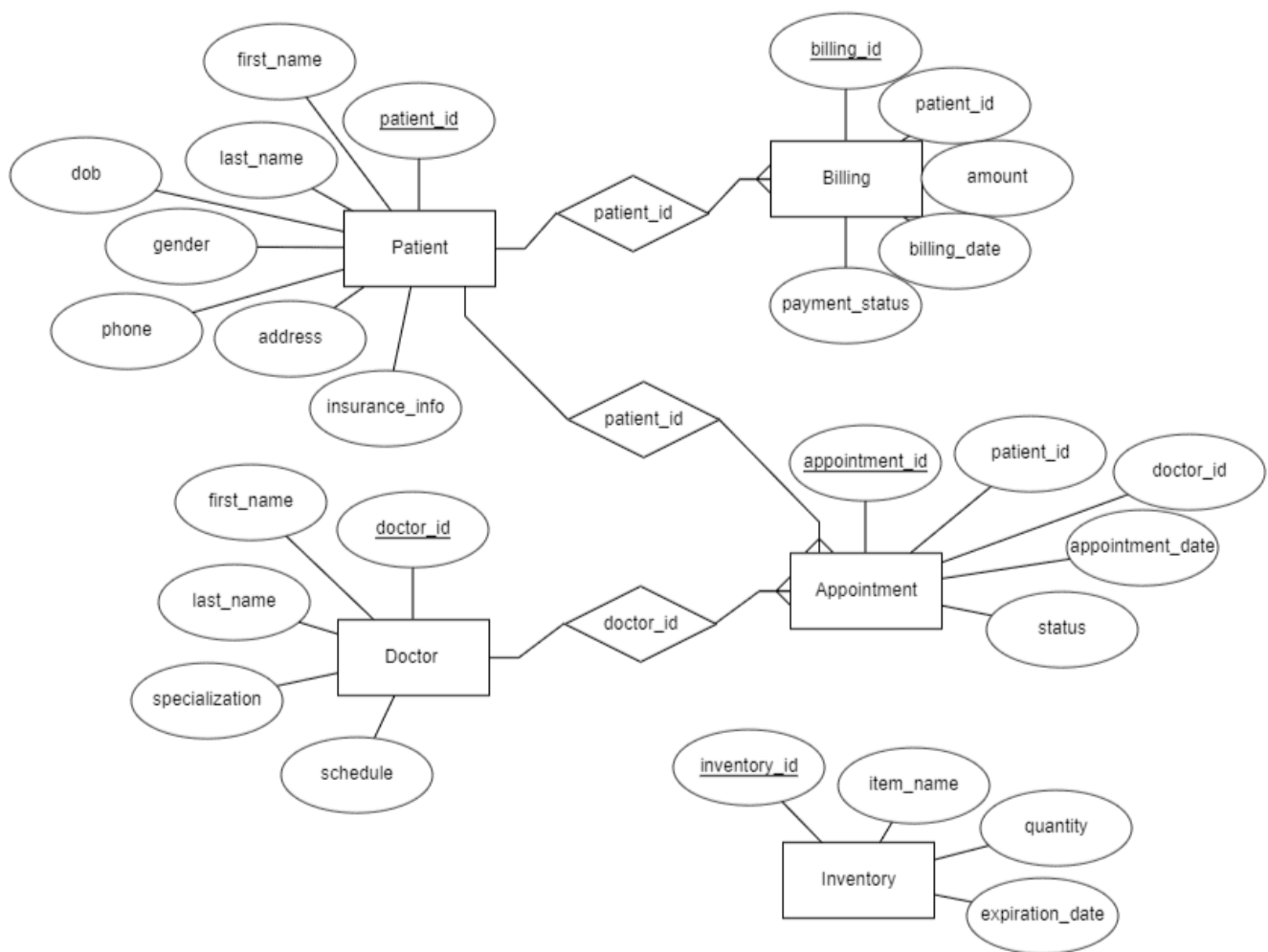
A. Administrative and Identification Entities:

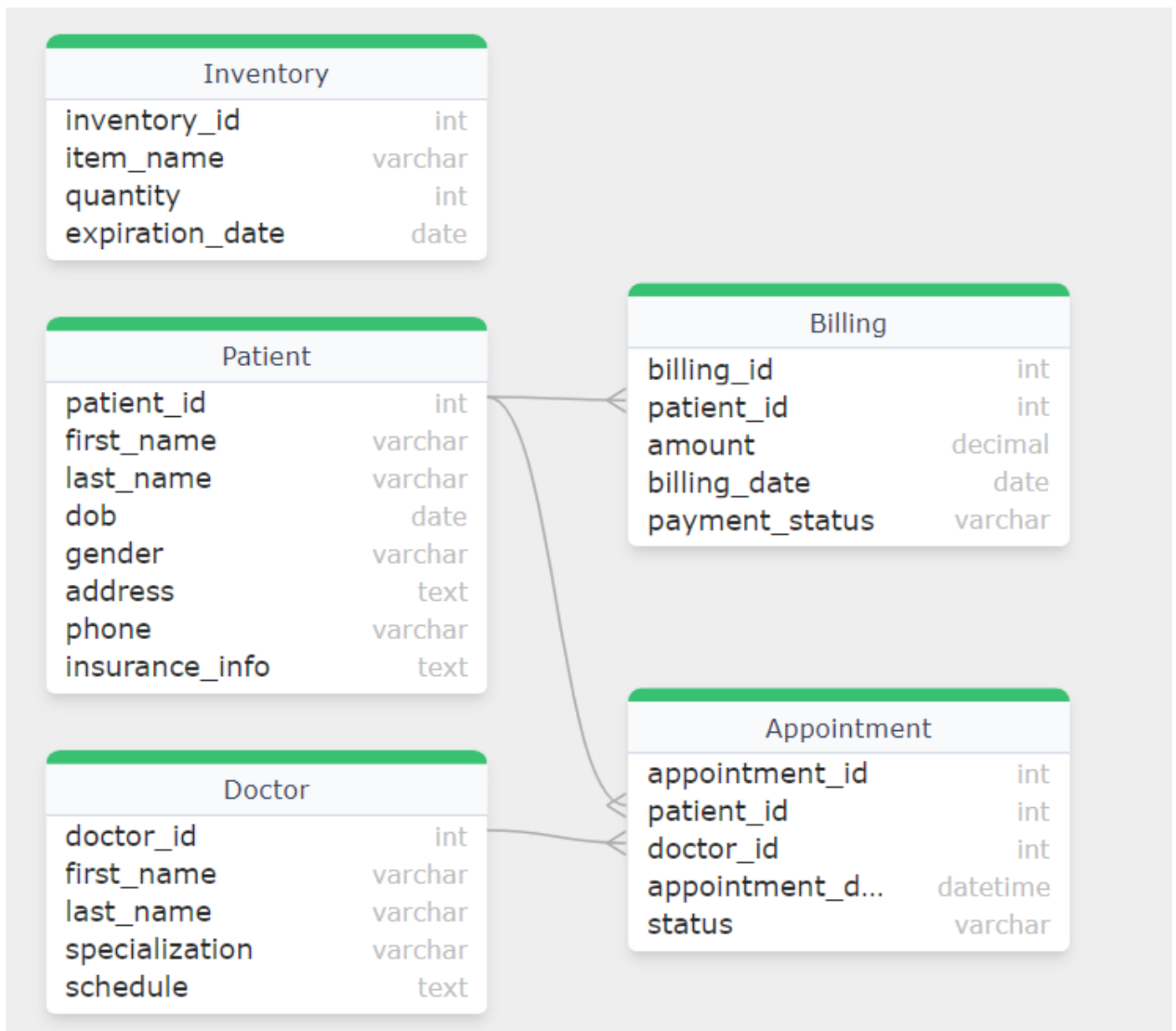
- **Patients Table:** Stores the unique personal profiles and permanent medical identities of each patient.
- **Doctors Table:** Manages medical staff information, specialties, and rosters.
- **System Users Table:** Controls access permissions across different operational roles (Admin, Receptionist, Laboratory Technician, Radiology Technician, Pharmacist).

B. Operational and Medical Entities:

- **Appointments Table:** Connects patients with doctors or clinical departments and coordinates schedules.

- **Prescriptions Table:** Represents the interactive medical hub that links the physician directly to the pharmacy, laboratory, and radiology departments.
- **Lab Results Table:** Stores quantitative data and uploads technical laboratory reports.
- **Radiology Results Table:** Manages diagnostic imaging summaries and links them directly to the patient's record.
- **Payments Table:** Documents and tracks financial transactions and invoice statuses.





4. Data Dictionary & Detailed Table Structures

Below is the precise technical breakdown for each table, defining fields, data types, and integrity constraints such as Primary Keys (PK) and Foreign Keys (FK):

1. Patients Table

This table handles the storage of patients' personal and medical demographics. The system utilizes **Automated ID Generation** to assign a unique identifier to each patient upon initial registration.

- **Patient_ID** (*int, Primary Key, Auto Increment*): The unique identifier for the patient.
- **Name** (*varchar(150), Not Null*): The full name of the patient.
- **Age** (*int*): The age of the patient.
- **Phone** (*varchar(20)*): The contact number used for communication and verification.

- Email (*varchar(100), Unique*): The patient's email address (integrated with the Gmail API for automated delivery of medical reports).
- Medical_History (*text*): Permanent medical notes, chronic illnesses, and historical health records.

2. Doctors Table

- Doctor_ID (*int, Primary Key, Auto Increment*): The unique identifier for the physician.
- Name (*varchar(150), Not Null*): The full name of the doctor.
- Specialization (*varchar(100)*): The medical specialty (e.g., Internal Medicine, Pediatrics, Surgery, Cardiology).
- Phone (*varchar(20)*): The doctor's contact number.
- Availability_Status (*boolean*): A flag indicating whether the doctor is currently available to accept new bookings.

```
CREATE TABLE `users` (
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `email` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `email_verified_at` timestamp NULL DEFAULT NULL,
  `password` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `remember_token` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `users_email_unique` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

3. Appointments Table

This table manages patient flow, optimizes waiting times, and dynamically prevents booking conflicts.

- **1. Design Philosophy**

The appointments table within the schema is not merely a passive record-keeper; it serves as the dynamic engine driving **Patient Flow Management** within the hospital. It is architected as a centralized relational hub that seamlessly bridges four core hospital domains: the Patient, the Doctor, the Department, and the Financial registry.

- **2. Architectural Columns Analysis**

The table columns can be logically classified into four functional categories based on their operational roles:

- **A. Relational Keys & Links**

id (bigint UNSIGNED): The auto-incrementing Primary Key. Utilizing bigint ensures the system can accommodate millions of appointments over years of hospital operations without running out of numerical bounds.

patient_id & doctor_id: These serve as Foreign Keys linking each appointment directly to the respective patient and doctor profiles, ensuring strict **Referential Integrity** across the database.

department_id: A strategic foreign key that routes the appointment to its specific department (e.g., Outpatient Clinics, Labs, Radiology), preventing scheduling silos.

- **B. Scheduling & Timing Constraints**

- **appointment_date (date):** Records the calendar date allocated for the appointment.

- **start_time & end_time (time):** This highlights the system's advanced scheduling logic. Instead of relying on vague time frames, it defines a precise time slot (e.g., 10:00 AM to 10:30 AM), effectively eliminating **Slot Overlapping** and scheduling conflicts.

- **queue_number (int):** Represents the sequential live queue position of the patient on that specific day, which forms the basis for the hospital's waiting room management.

- **C. Operational Workflow & Status Trackers**

-**status (varchar):** Manages the entire **Appointment Lifecycle**. Every entry initiates as 'pending' and transitions through system-defined operational states (e.g., 'confirmed', 'cancelled', 'completed').

-**checked_in_at (timestamp):** Captures the exact moment the patient physically arrives at the reception desk to check in. By querying the delta between checked_in_at and start_time, the system can compute critical key performance indicators (KPIs) like **Patient Lateness Rates** and **Average Wait Times**.

-**auto_confirmed_at:** Dedicated to automated system triggers that confirm bookings instantly without requiring manual human intervention once specific criteria are met.

- **D. Medical & Financial Records**

-**treatment:** Explicitly logs the clinical purpose of the visit (e.g., "Cardiac Consultation").

-**notes (text)**: A flexible, long-form text field designed to capture initial patient complaints or clinical remarks. It fully supports Arabic characters due to the global utf8mb4 character encoding applied in the database dump.

-**fee (decimal(10,2))**: A high-precision financial column tracking the cost of the appointment. Choosing decimal instead of float or double is an industry standard that guarantees zero rounding errors during financial reporting.

- **3. The Appointment Data Lifecycle**

-Based on the transactional data within the SQL dump, the system processes each appointment through a structured pipeline:

-**Creation Stage**: The record is initialized with automated created_at timestamps, defaulting the status to 'pending'.

-**Arrival Stage (Check-in)**: Upon physical arrival, the receptionist updates the record, triggering the checked_in_at timestamp and assigning a sequential queue_number.

-**Closure Stage**: The transactional fee is finalized, and the status shifts to its final operational state (e.g., 'completed').

```
CREATE TABLE `appointments` (  
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,  
  `patient_id` bigint UNSIGNED NOT NULL,  
  `doctor_id` bigint UNSIGNED NOT NULL,  
  `department_id` bigint UNSIGNED DEFAULT NULL,  
  `appointment_date` date NOT NULL,  
  `start_time` time NOT NULL,  
  `end_time` time NOT NULL,  
  `status` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'pending',  
  `queue_number` int UNSIGNED DEFAULT NULL,  
  `auto_confirmed_at` timestamp NULL DEFAULT NULL,  
  `checked_in_at` timestamp NULL DEFAULT NULL,  
  `confirmation_email_sent_at` timestamp NULL DEFAULT NULL,  
  `treatment` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `notes` text COLLATE utf8mb4_unicode_ci,  
  `phone_number` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  `fee` decimal(10,2) NOT NULL DEFAULT '0.00',  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4. Prescriptions Table

This table acts as the cornerstone of the "Electronic Prescription System." Once saved by a doctor, the record instantly routes data to the pharmacy and diagnostic labs.

- *Prescription_ID (int, Primary Key, Auto Increment)*: The unique prescription ID.
- *Doctor_ID (int, Foreign Key references Doctors)*: The prescribing physician.
- *Patient_ID (int, Foreign Key references Patients)*: The target patient.
- *Diagnosis_Notes (text)*: Clinical diagnosis notes recorded during the examination.
- *Medication (text)*: Detailed prescribed drugs, dosages, and administration schedules.
- *Lab_Required (text)*: Requested laboratory tests (if applicable).
- *Radiology_Required (text)*: Requested diagnostic imaging or scans (if applicable).
- *Issue_Date (timestamp)*: The exact timestamp of creation.

```
CREATE TABLE `diagnoses` (  
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,  
  `appointment_id` bigint UNSIGNED NOT NULL,  
  `patient_id` bigint UNSIGNED NOT NULL,  
  `doctor_id` bigint UNSIGNED NOT NULL,  
  `symptoms` text COLLATE utf8mb4_unicode_ci NOT NULL,  
  `diagnosis` text COLLATE utf8mb4_unicode_ci NOT NULL,  
  `prescription` text COLLATE utf8mb4_unicode_ci,  
  `treatment_plan` text COLLATE utf8mb4_unicode_ci,  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  KEY `diagnoses_appointment_id_foreign` (`appointment_id`),  
  KEY `diagnoses_patient_id_foreign` (`patient_id`),  
  KEY `diagnoses_doctor_id_foreign` (`doctor_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

5. Lab Results Table

- *Result_ID (int, Primary Key, Auto Increment)*: The unique test result identifier.
- *Patient_ID (int, Foreign Key references Patients)*: The patient tested.
- *Prescription_ID (int, Foreign Key references Prescriptions)*: The reference prescription that ordered the lab work.

- Test_Name (*varchar(100)*): The medical name of the panel or test (e.g., CBC, Liver Function Tests).
- Result_Value (*text*): The lab values, findings, or textual diagnostic report.
- Uploaded_File_Path (*varchar(255)*): The directory path of the uploaded report file (PDF) for direct cloud access and viewing.
- Upload_Date (*timestamp*): The timestamp of when the technician uploaded the results.

6. Radiology Results Table

- Radiology_ID (*int, Primary Key, Auto Increment*): The unique radiology report ID.
- Patient_ID (*int, Foreign Key references Patients*): The patient examined.
- Test_Name (*varchar(100)*): The imaging modality used (e.g., X-Ray, MRI, CT Scan).
- Report_Details (*text*): The professional diagnostic narrative written by the radiologist or technician.
- File_Path (*varchar(255)*): The digital storage path for the scan image or digital report.
- Date (*timestamp*): The exact time of the radiological procedure.

```
CREATE TABLE `scan_results` (
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,
  `scan_request_id` bigint UNSIGNED NOT NULL,
  `patient_id` bigint UNSIGNED NOT NULL,
  `doctor_id` bigint UNSIGNED DEFAULT NULL,
  `entered_by_id` bigint UNSIGNED DEFAULT NULL,
  `findings` text COLLATE utf8mb4_unicode_ci NOT NULL,
  `impression` text COLLATE utf8mb4_unicode_ci,
  `scan_image` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `scan_results_scan_request_id_foreign` (`scan_request_id`),
  KEY `scan_results_patient_id_foreign` (`patient_id`),
  KEY `scan_results_doctor_id_foreign` (`doctor_id`),
  KEY `scan_results_entered_by_id_foreign` (`entered_by_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

7. Payments Table

Designed to eliminate processing delays and revenue tracking errors common in traditional, manual hospital billing systems.

- **Payment_ID** (*int, Primary Key, Auto Increment*): The unique financial transaction ID.
- **Patient_ID** (*int, Foreign Key references Patients*): The patient responsible for payment settlement.
- **Amount** (*decimal(10,2)*): The precise monetary amount owed/paid.
- **Payment_Purpose** (*varchar(150)*): The reason for the invoice (e.g., Consultation Fee, Lab Test, Scan).
- **Status** (*enum: 'Paid', 'Pending', 'Refunded'*): The financial lifecycle status of the transaction.
- **Transaction_Date** (*timestamp*): The timestamp when the payment transaction was officially processed.

5. Relationships & Integrity Constraints

To eliminate structural anomalies and prevent the loss of critical medical records (**Data Integrity**), the following engineering rules have been enforced at the schema level:

- **One-to-Many (1:N) Relationship between Patients and Operational Data (Appointments / Prescriptions / Payments)**: A single patient (**Patient_ID**) can book multiple appointments, own an extensive history of medical prescriptions, and execute various financial transactions over time. However, any individual booking, prescription, or invoice must belong strictly to one specific patient.
- **One-to-Many (1:N) Relationship between Doctors and Operational Data (Appointments / Prescriptions)**: An individual physician can manage an array of scheduled appointments, consult multiple patients, and issue numerous independent electronic prescriptions.
- **Referential Action Control (ON UPDATE CASCADE / ON DELETE RESTRICT)**: The system completely restricts (**RESTRICT**) the deletion of any doctor or patient record if it is linked to existing medical or financial rows. This strictly complies with healthcare regulations demanding the indefinite preservation of patient health histories. Conversely, if a primary user ID is modified, the changes

automatically propagate (CASCADE) through all relational tables to maintain structural unity.

```
-- 1. Constraints for table: scan_requests
ALTER TABLE `scan_requests`
  ADD CONSTRAINT `scan_requests_diag_fk`
    FOREIGN KEY (`diagnosis_id`)
    REFERENCES `diagnoses` (`id`)
    ON DELETE CASCADE,

  ADD CONSTRAINT `scan_requests_doc_fk`
    FOREIGN KEY (`doctor_id`)
    REFERENCES `doctors` (`id`)
    ON DELETE SET NULL,

  ADD CONSTRAINT `scan_requests_pat_fk`
    FOREIGN KEY (`patient_id`)
    REFERENCES `patients` (`id`)
    ON DELETE CASCADE;

-- 2. Constraints for table: scan_results
ALTER TABLE `scan_results`
  ADD CONSTRAINT `scan_results_req_fk`
    FOREIGN KEY (`scan_request_id`)
    REFERENCES `scan_requests` (`id`)
    ON DELETE CASCADE,

  ADD CONSTRAINT `scan_results_pat_fk`
    FOREIGN KEY (`patient_id`)
    REFERENCES `patients` (`id`)
    ON DELETE CASCADE,

  ADD CONSTRAINT `scan_results_doc_fk`
    FOREIGN KEY (`doctor_id`)
    REFERENCES `doctors` (`id`)
    ON DELETE SET NULL,

  ADD CONSTRAINT `scan_results_user_fk`
    FOREIGN KEY (`entered_by_id`)
    REFERENCES `users` (`id`)
    ON DELETE SET NULL;
```

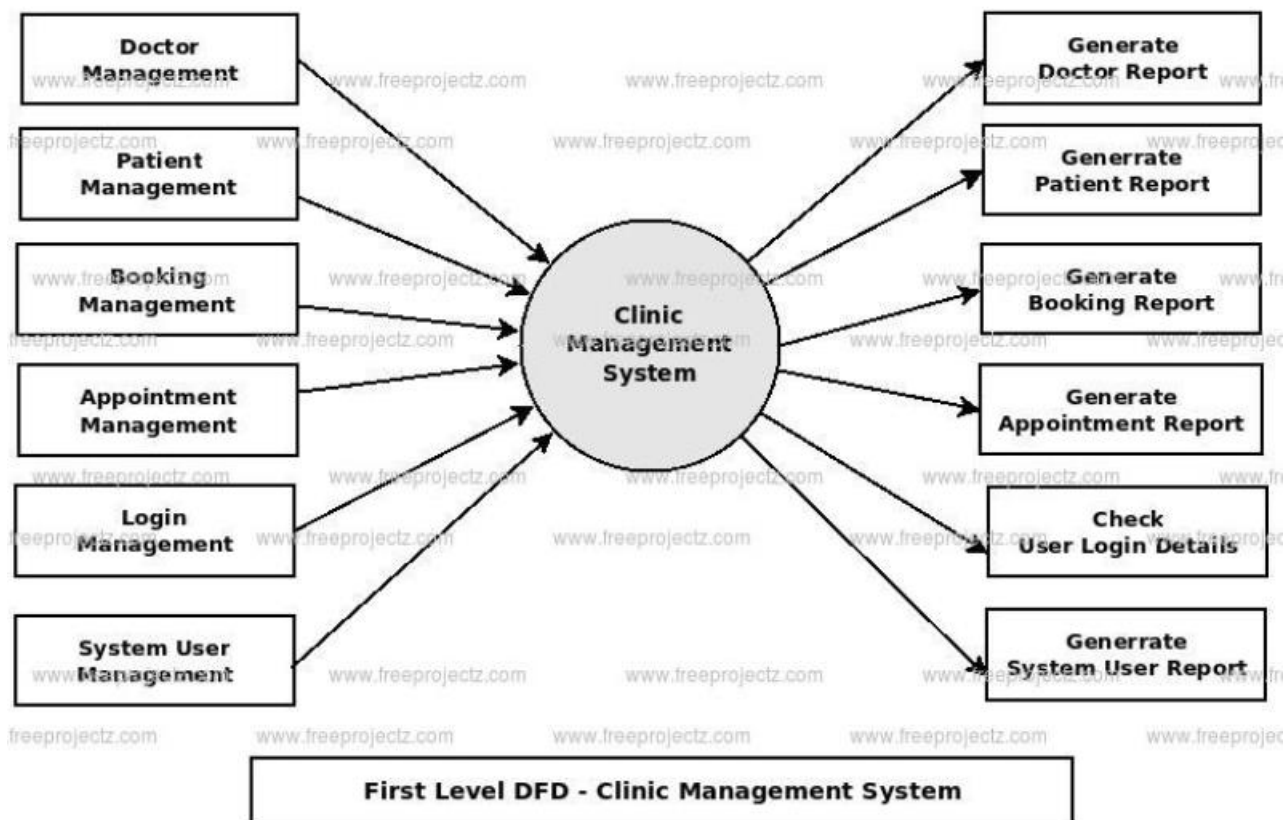
6. Data Workflow Execution

Data moves dynamically through the relational schema following the logical sequence of a patient's journey within the smart facility:

1. **The Booking Stage:** The patient sets up an account. The Patients table captures the details and automatically triggers an ID

generation. Simultaneously, a corresponding record is inserted into the Appointments table.

2. **The Examination Stage:** The physician queries the patient's record from the Patients table via their unique ID. Following the check-up, the clinical diagnosis, medication list, and diagnostic orders are stored within the Prescriptions table.
3. **The Integration and Distribution Stage:** Immediately upon the insertion of a prescription record, the pharmacy, lab, and radiology interfaces actively listen via targeted SQL queries to read outstanding medication or testing orders in real time.
4. **The Financial and Medical Update Stage:** The laboratory updates the system by writing data to the Lab Results table, immediately making it visible on the patient's portal, while the reception desk updates the status of the Payments table upon invoice settlement.

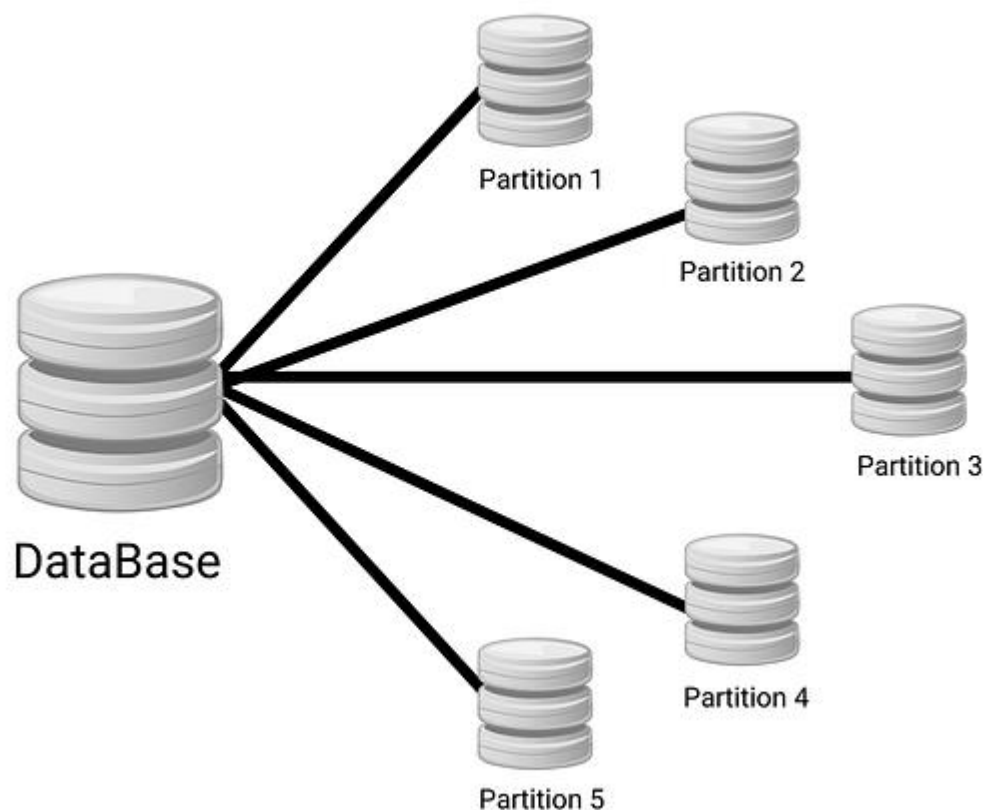


5. Future Database Enhancements (التطوير المستقبلي لقاعدة البيانات)

To ensure the scalability, high availability, and long-term efficiency of the hospital management system as it transitions from a localized prototype to an enterprise-grade medical solution, several advanced database methodologies will be integrated in future phases:

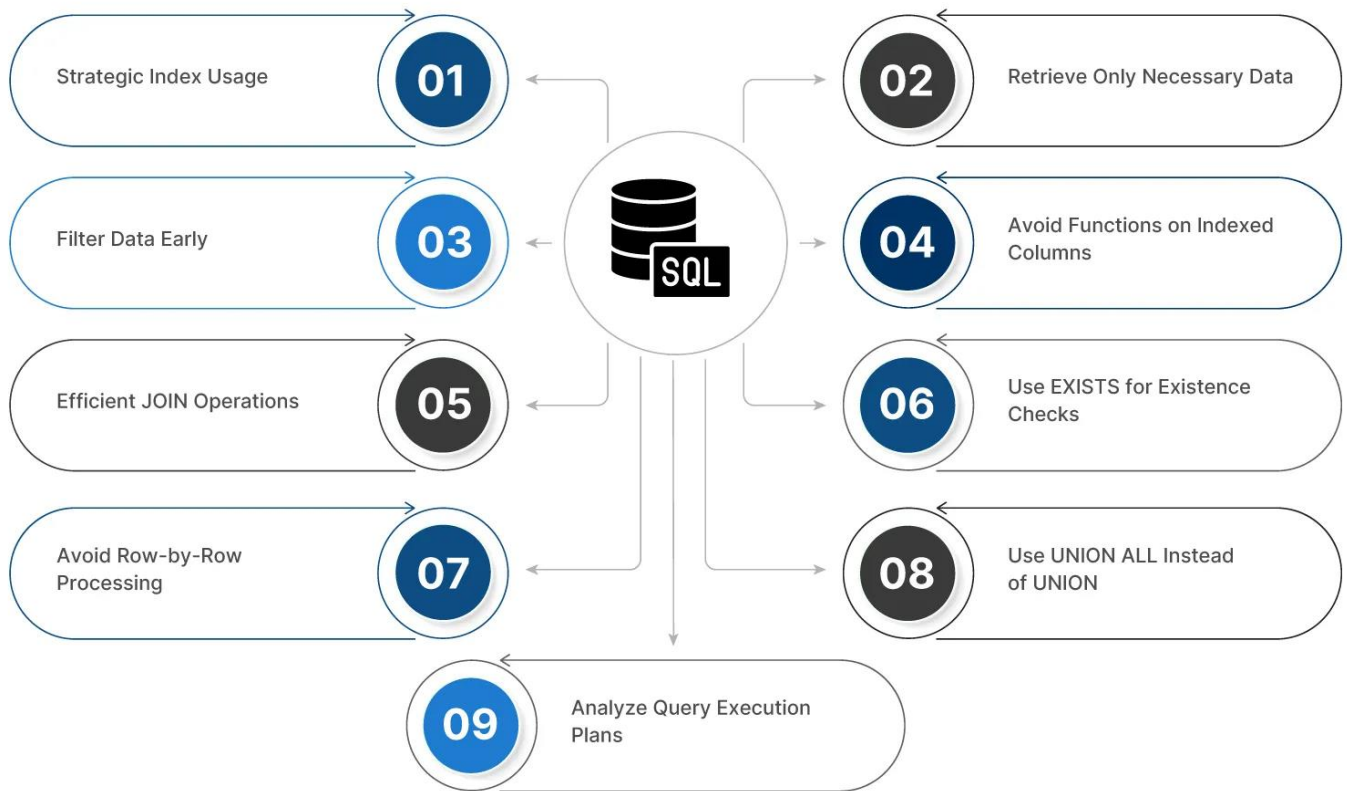
A. Database Partitioning & Sharding

As the hospital expands and the appointments table accumulates millions of transactional records over time, performance bottlenecks could arise during standard queries. To mitigate this, **Horizontal Partitioning (Sharding)** will be implemented. For instance, the appointments table can be partitioned by `appointment_date` (e.g., creating separate physical partitions for each calendar year). This dramatically reduces query execution time and optimizes index lookups.



B. Index Optimization for High-Frequency Queries

To accommodate rapid real-time lookups at the hospital reception desk, custom composite indexes will be deployed. Creating indexes on combined columns such as `(doctor_id, appointment_date)` or `(patient_id, status)` will ensure that the system fetches operational data with sub-millisecond latency, even during peak hospital hours.



C. Advanced Archiving Strategy & Data Warehousing

To maintain a lean and fast production database, an automated archiving pipeline will be developed. Historical data (e.g., completed or cancelled appointments older than three years) will be automatically migrated to a dedicated **Data Warehouse (OLAP system)**. This preserves historical medical records for long-term analytics and reporting without slowing down day-to-day hospital operations.

D. Multi-Tenant Architecture & Data Isolation

If the application expands to serve a network of multiple hospital branches or clinics, the database schema will be upgraded to a **Multi-Tenant Architecture**. This will be achieved either through a "Shared Database, Separate Schemas" approach or by introducing a strict `branch_id` tenant identifier across all operational tables, enforcing absolute data isolation and security between different healthcare facilities.

E. Replication and High Availability (HA)

To achieve zero-downtime and robust disaster recovery, a **Master-Slave Replication** topology will be configured. The Master database will handle all write operations (such as creating new appointments or processing payments), while one or more Slave databases will handle read-heavy operations (like generating management reports or viewing patient histories), ensuring continuous system availability even during hardware failures.